

```

BARSV_APP_Tkinter/
|
|— main.py                # Punctul de intrare al aplicației Tkinter
|
|— core/                  # Modul pentru logica de bază, non-GUI
|   |— __init__.py
|   |— security.py        # Funcții pentru verificările de securitate (hostname, user)
|   |— doc_generator.py   # Logica pentru generarea documentelor .docx (folosind docxtpl)
|   |— backup_manager.py  # Funcții pentru salvarea/încărcarea backup-urilor (folosind pandas)
|   |— utils.py           # Utilitare diverse (ex: handler caractere românești, formatare date)
|
|— ui/                    # Modul pentru componentele interfeței grafice Tkinter
|   |— __init__.py
|   |— app_window.py      # Clasa principală a ferestrei aplicației (structura generală, meniu, tab-uri principale)
|   |— tabs/              # Subdirector pentru modulele fiecărui tab principal
|       |— __init__.py
|       |— principal_tab.py # UI pentru tab-ul 'Principal'
|       |— auto_tab.py     # UI pentru tab-ul 'Auto Implicate' (include logica de adăugare dinamică)
|       |— victime_tab.py  # UI pentru tab-ul 'Martori/Victime' (include logica de adăugare dinamică victime/martori)
|       |— cfl_tab.py      # UI pentru tab-ul 'CFL'
|       |— backup_tab.py   # UI pentru tab-ul 'Backup Database'
|       |— documents_tab.py # UI pentru tab-ul '## Documente ##' (selecție template-uri)
|   |— widgets/           # Subdirector pentru widget-uri custom sau reutilizabile
|       |— __init__.py
|       |— scrollable_frame.py # O clasă reutilizabilă pentru a crea cadre cu scrollbar
|       |— victim_frame.py   # Funcție/Clasă pentru a genera un cadru de victimă în Tkinter
|       |— auto_frame.py     # Funcție/Clasă pentru a genera un cadru auto+șofer în Tkinter
|
|— templates/             # Director pentru șabloanele .docx
|   |— #Ordonanta_CFL_Template.docx
|   |— #Ordonanta_IUP_Template.docx
|   |— #Template_CFL_1_auto.docx
|   |— #Template_CFL_2_auto.docx
|   |— #Template_CFL_3_auto.docx
|   |— ... (toate celelalte fișiere .docx necesare)
|
|— requirements.txt        # Fișierul cu dependențele Python (pandas, python-docx, docxtpl, etc.)

```

Rezumatul Conținutului:

1. **BARSV_APP_Tkinter/ (Director Rădăcină):** Conține întreaga aplicație.
2. **main.py :**
 - Punctul de start al aplicației.
 - Importă și inițializează fereastra principală din `ui.app_window`.
 - Efectuează verificările inițiale de securitate apelând funcții din `core.security`.
 - Pornește bucla principală de evenimente Tkinter (`root.mainloop()`).
3. **core/ :**
 - **security.py :** Păstrează funcțiile `get_local_ip`, `get_local_hostname`, `get_logged_in_user` și logica de verificare a permisiunilor. Eventualele mesaje de eroare vor folosi `tkinter.messagebox`.
 - **doc_generator.py :** Conține funcția adaptată `saveFile` (poate redenumită `generate_documents`). Va primi datele de la interfața Tkinter (probabil sub forma unui dicționar structurat), va interacționa cu `tkinter.filedialog` pentru selectarea folderului și va folosi `docxtpl` pentru a genera documentele pe baza template-urilor din folderul `templates/`.
 - **backup_manager.py :** Implementează funcționalitățile de autosave (cu `threading` și `time`), salvare manuală și încărcare backup folosind `pandas` pentru a citi/scrie fișiere `.xls` / `.xlsx`. Va interacționa cu `tkinter.filedialog` și `tkinter.messagebox`. Include și funcția de refresh listă backup.
 - **utils.py :** Conține funcția adaptată pentru caracterele românești (va trebui legată la evenimentele `KeyPress` ale widget-urilor `tkinter.Entry` și `tkinter.Text`) și alte posibile funcții utilitare.
4. **ui/ :**
 - **app_window.py :** Definește clasa ferestrei principale (`tk.Tk` sau `tk.Toplevel`). Creează structura de bază, inclusiv `ttk.Notebook` pentru tab-uri și instanțiază clasele/apellează funcțiile corespunzătoare din `ui.tabs/` pentru a popula fiecare tab.
 - **tabs/ :** Fiecare fișier (`principal_tab.py`, `auto_tab.py`, etc.) va conține codul Tkinter specific pentru construirea interfeței aceluia tab.
 - Vor folosi widget-uri `tkinter` și `tkinter.ttk` (ex: `ttk.Label`, `ttk.Entry`, `ttk.Combobox`, `tk.Text`, `ttk.Checkbutton`, `ttk.Radiobutton`, `ttk.Button`).
 - Pentru layout, se va folosi predominant managerul `grid`, care este potrivit pentru formulare complexe. `pack` poate fi util pentru elemente simple sau butoane la bază.
 - Logica de adăugare dinamică (vehicule, victime) va implica crearea de noi instanțe de cadre (din `ui.widgets/`) și adăugarea lor în layout-ul corespunzător (probabil într-un `ScrollableFrame`).

- `widgets/` :
 - `scrollable_frame.py` : O componentă esențială pentru a recrea funcționalitatea `sg.Column(scrollable=True)` . De obicei, implică un `tk.Canvas` , un `ttk.Frame` interior pe canvas și `ttk.Scrollbar` .
 - `victim_frame.py` , `auto_frame.py` : Vor conține funcții sau clase care returnează un `ttk.Frame` populat cu widget-urile necesare pentru o victimă sau un vehicul/șofer, similar cu funcțiile `generate_victim_frame` și `generate_auto_frame` din codul original. Acestea vor facilita adăugarea dinamică.
- 5. `templates/` : Conține *exact aceleași* fișiere șablon `.docx` pe care le folosește aplicația PySimpleGUI. Calea către acest director va trebui specificată corect în `core.doc_generator.py` .
- 6. `requirements.txt` : Listează toate pachetele Python externe necesare: `pandas` , `python-docx` , `docxtpl` , `pyperclip` , `openpyxl` .

Această structură promovează separarea responsabilităților (UI vs. logică), făcând codul mai ușor de înțeles, modificat și întreținut. Conversia va fi un proces laborios, necesitând atenție la maparea widget-urilor, gestionarea layout-ului și adaptarea logicii de evenimente și colectare a datelor la specificul Tkinter.